

République du Sénégal



Un Peuple – Un But – Une Foi

Ministère de l'Enseignement Supérieur et de la Recherche

Université Alioune DIOP de Bambey



UFR Sciences Appliquées et Technologies de l'Information et de la
Communication (SATIC)

Département Technologies de l'Information et de la Communication (TIC)

Master professionnelle
Systèmes D'Information (S.I)

PROJET DE MÉMOIRE

**Déploiement d'une Application
Web Répartie avec Docker,
Kubernetes, Ansible et Jenkins**

Encadrant(s) :

M. Youssou LAM

Présenté par :

Adama Seck

Année académique 2025-2026

Introduction

1.1 Contexte

Dans le cadre du module DevOps et Systèmes Répartis, nous avons réalisé un projet complet consistant à concevoir, développer et déployer une application web distribuée.

L'objectif était de mettre en œuvre :

- Une architecture microservices moderne
- Une conteneurisation avec Docker
- Un déploiement sur Kubernetes
- Une automatisation avec Ansible
- Un pipeline CI/CD complet avec Jenkins

Ce projet simule le besoin d'une startup technologique souhaitant déployer une plateforme web maintenable et scalable.

Architecture Générale

2.1 Vue d'ensemble

L'application est composée de trois services :

- Frontend (React)
- Backend (Flask API REST)
- Base de données PostgreSQL

Architecture logique

Utilisateur → Frontend (React) → Backend API (Flask) → PostgreSQL

2.2 Architecture DevOps complète

GitHub → Jenkins → (CI/CD) → Docker Build → Docker Hub → Kubernetes (Minikube) → Pods (Frontend / Backend / Postgres)

Choix Techniques

3.1 Frontend

- React (Vite)
- Axios
- JWT Authentication
- Framer Motion

Choix justifié par :

- Rapidité de développement
- Architecture composant
- Large écosystème

3.2 Backend

- Flask
- SQLAlchemy
- JWT Authentication
- Flask-Migrate

Flask a été choisi pour :

- Sa légèreté
- Sa simplicité
- Son intégration facile avec PostgreSQL

3.3 Base de données

- PostgreSQL 15
- Volume persistant Kubernetes (PVC)

Choisie pour :

- Robustesse
- Support transactions
- Standard entreprise

Conteneurisation (Docker)

4.1 Dockerfiles

- Dockerfile Backend
- Dockerfile Frontend

Chaque service possède :

- Image dédiée
- Port exposé
- Variables d'environnement

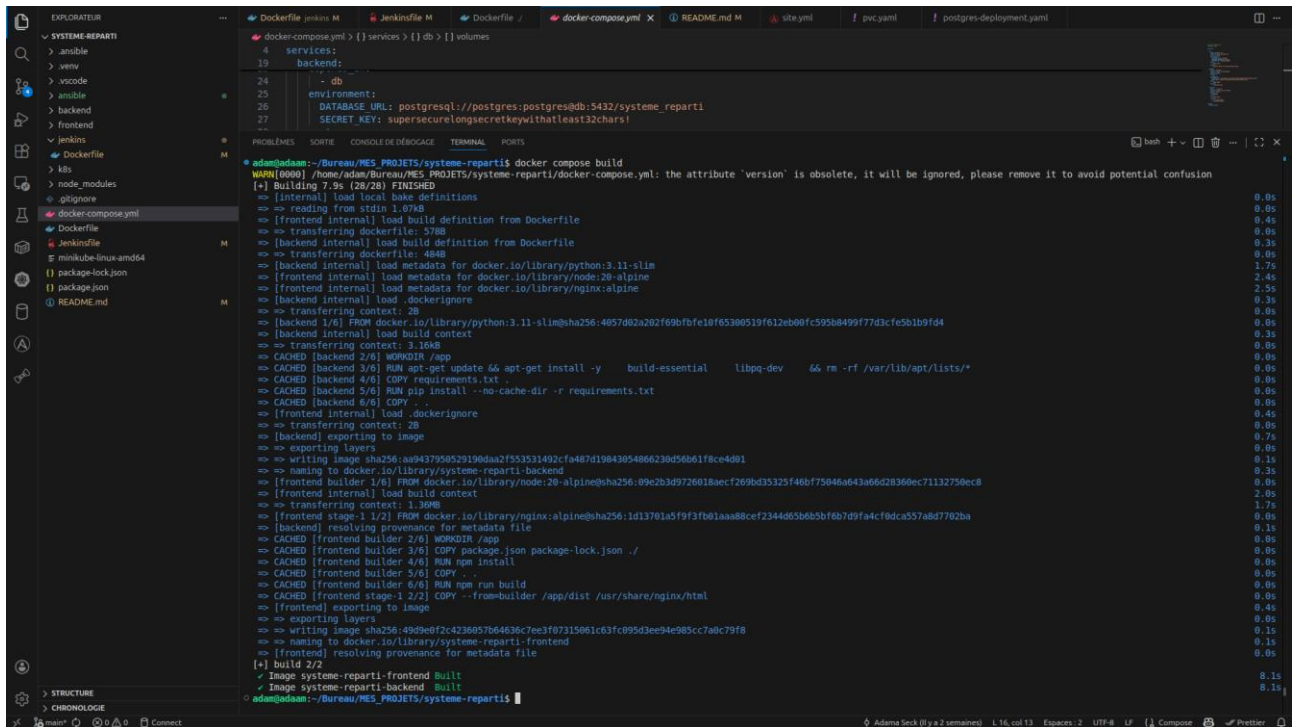
4.2 Docker Compose

Utilisé pour :

- Test local multi-services
- Simulation environnement production

Commande :

docker-compose up



```
adam@adam:~/Bureau/ME5_PROJETS/systeme-repartis$ docker-compose build
WARN[0000] /home/adam/Bureau/ME5_PROJETS/systeme-repartis/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 7.9s (28/28) FINISHED
=> [internal] load local bake definitions
=> [frontend internal] load build definition from Dockerfile
=> == transferring dockerfile: 578B
=> [backend internal] load build definition from Dockerfile
=> == transferring dockerfile: 484B
=> [backend internal] load metadata for docker.io/library/python:3.11-slim
=> [frontend internal] load metadata for docker.io/library/node:20-alpine
=> [backend internal] load metadata for docker.io/library/nginx:alpine
=> [backend internal] load .dockerignore
=> == transferring context: 2B
=> [backend 1/6] FROM docker.io/library/python:3.11-slim@sha256:4057d92a202f09bf0e18f65306519f612eb00fc595b8499f77d3cfe5b1b9f64
=> == transferring context: 3.16kB
=> CACHED [backend 2/6] WORKDIR /app
=> CACHED [backend 3/6] RUN apt-get update && apt-get install -y build-essential libpq-dev && rm -rf /var/lib/apt/lists/*
=> CACHED [backend 4/6] COPY requirements.txt
=> CACHED [backend 5/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [backend 6/6] COPY .
=> [frontend internal] load .dockerignore
=> == transferring context: 2B
=> [backend] exporting to image
=> == exporting layers
=> == writing image sha256:aa9437950529190aa2f55351492cf0487d19843054866230d5661f8ce4d01
=> == naming to docker.io/library/systeme-repartis-backend
=> [frontend builder 1/6] FROM docker.io/library/node:20-alpine@sha256:09e2b3d9726018aecf269bd33325f40bf75046a643a66d28360ec71132750ec8
=> [backend internal] load build context
=> == transferring context: 1.36kB
=> [frontend stage-1 1/2] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344065b6b5bf607d9fa4cf0dc557a0d7702ba
=> [backend] resolving provenance for metadata file
=> CACHED [frontend builder 2/6] WORKDIR /app
=> CACHED [frontend builder 3/6] COPY package-lock.json
=> CACHED [frontend builder 4/6] RUN npm install
=> CACHED [frontend builder 5/6] COPY .
=> CACHED [frontend builder 6/6] RUN npm run build
=> CACHED [frontend stage-1 2/2] COPY --from=builder /app/dist /usr/share/nginx/html
=> [frontend] exporting to image
=> == exporting layers
=> == writing image sha256:49d9e0f2c4236957b64636c7ee3f07315061c63fc095d3ee94e985cc7a0c79f8
=> == naming to docker.io/library/systeme-repartis-frontend
=> [frontend] resolving provenance for metadata file
[+] build 2/2
Image system-repartis-frontend Built
Image system-repartis-backend Built
adam@adam:~/Bureau/ME5_PROJETS/systeme-repartis$
```

docker-compose build

5.1 Manifests créés

- backend-deployment.yaml
- frontend-deployment.yaml
- postgres-deployment.yaml
- services associés
- persistentvolumeclaim.yaml

5.2 Fonctionnalités

- Déploiement ReplicaSet
- Exposition via NodePort
- Volume persistant PostgreSQL
- Rolling update automatique

Commande principale :

```
kubectl apply -f k8s/
```

Vérification :

```
kubectl get pods
```

```
kubectl get svc
```

```
Jenkinsfile
1 pipeline {
2   agent any
3 }
4 environment {
5   ...
6 }

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ minikube status
minikube
type: Control Plane
host: Stopped
kubelet: Stopped
apiserver: Stopped
kubeconfig: Stopped

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ minikube start
minikube v1.35.1 sur Ubuntu 24.04
Kubernetes 1.35.1 est désormais disponible. Si vous souhaitez effectuer une mise à niveau, spécifiez --kubernetes-version=v1.35.1
Utilisation du pilote docker basé sur le profil existant
Démarrage du nœud "minikube" primary control-plane dans le cluster "minikube"
Extraction de l'image de base v0.0.50...
Redémarrage du docker container existant pour "minikube" ...
Préparation de Kubernetes v1.35.0 sur containerd 2.7.1...
Utilisation de l'image gcr.io/k8s-minikube/storage-provisioner:v5
Vérification des composants Kubernetes...
Modules actifs: default-storageclass, storage-provisioner
Terminé ! kubectl est maintenant configuré pour utiliser "minikube" cluster et espace de noms "default" par défaut.

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ kubectl apply -f .
deployment.apps/backend unchanged
service/backend unchanged
deployment.apps/frontend unchanged
service/frontend unchanged
deployment.apps/postgres unchanged
service/postgres unchanged
persistentvolumeclaim/postgres-pvc unchanged

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
backend-7f946f9f45-whfn4   1/1     Running   4 (2m4s ago)    9d
frontend-5f5d7f6cc7-6n9lf  1/1     Running   5 (2m4s ago)    13d
postgres-57579b5dc-bvnmj   1/1     Running   4 (2m4s ago)    9d

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
minikube            Ready    control-plane   12d   v1.35.0

(.venv) adam@adam:~/Bureau/MES_PROJETS/systeme-reparti/k8s$ kubectl get svc
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP    PORT(S)          AGE
backend             NodePort      10.98.207.105    <none>          5000/TCP          13d
frontend            NodePort      10.98.21.177     <none>          80/TCP            13d
kubernetes           ClusterIP     10.98.0.1        <none>          443/TCP           13d
postgres            ClusterIP     None            <none>          5432/TCP          13d
```

Automatisation avec Ansible

Un playbook `site.yml` a été développé pour :

- Installer Docker
- Installer kubectl
- Installer Minikube
- Lancer Jenkins en conteneur
- Déployer automatiquement l'application

Exécution :

```
ansible-playbook -i inventory.ini site.yml --ask-become-pass
```

Cela permet un provisionnement complet en une seule commande.

CI/CD avec Jenkins

Jenkins est exécuté en container Docker.

Pipeline automatisé :

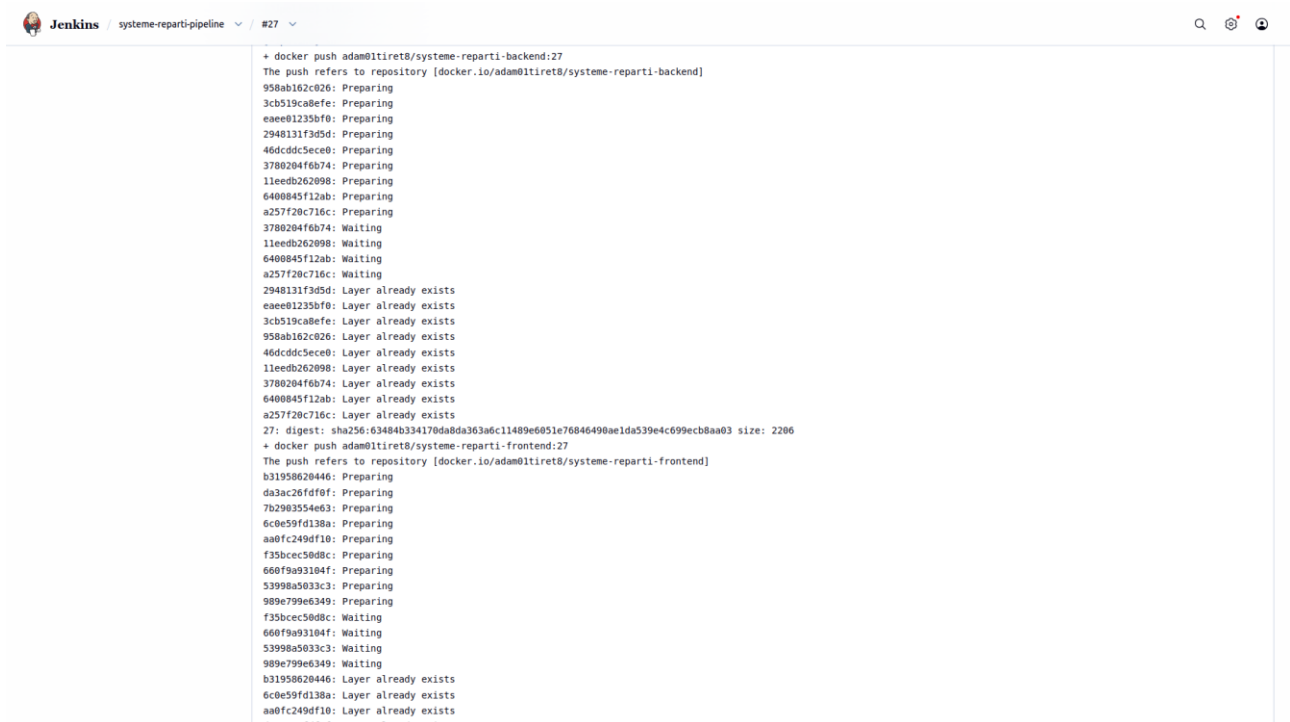
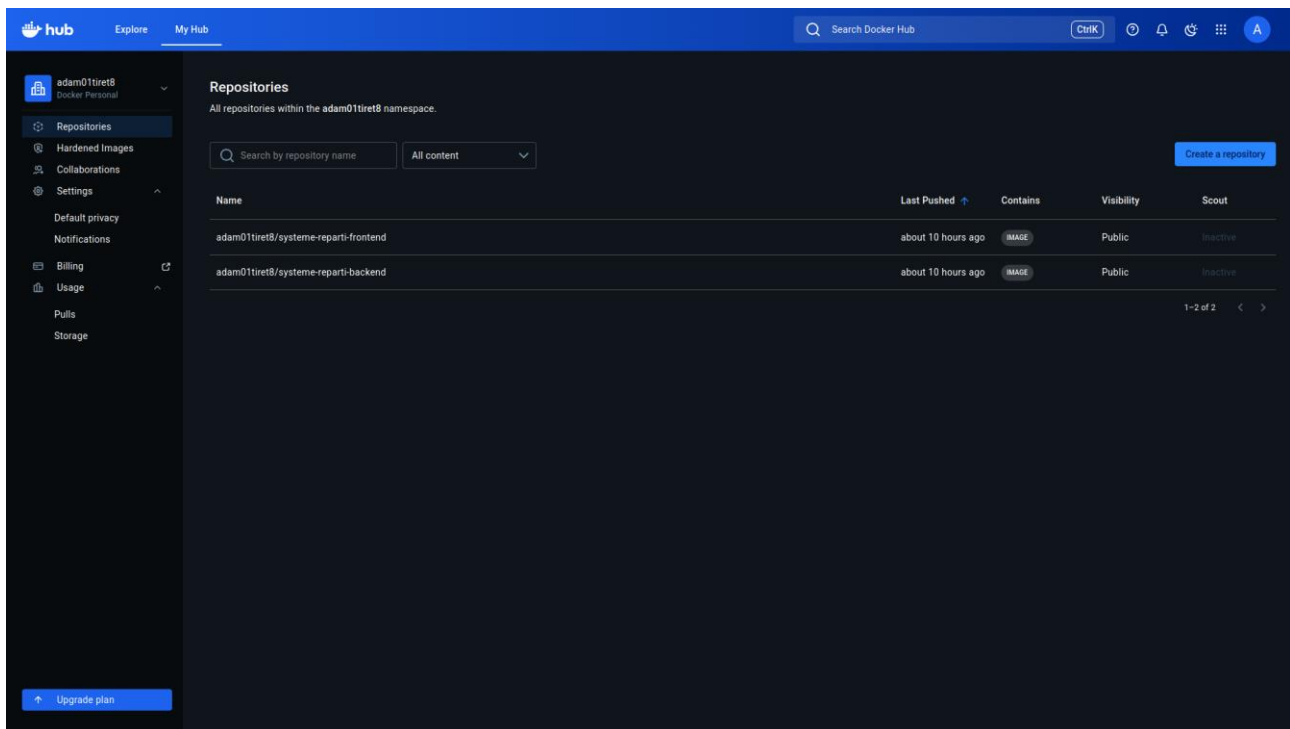
1. Checkout GitHub
2. Lint & Tests
3. Build images Docker
4. Push Docker Hub
5. Déploiement Kubernetes

Architecture :

Git push → Jenkins → Docker → Docker Hub → Kubernetes

Résultat :

- Déploiement sans intervention manuelle
- Mise à jour automatique des pods
- Rolling update sécurisé



Sécurité & Bonnes Pratiques

- Variables d'environnement pour secrets
- Credentials Docker Hub stockés dans Jenkins
- Pas de commit des fichiers .env
- Isolation via containers

- Volume persistant pour base de données

Tests & Validation

Tests réalisés :

- Test API via `/api/ping`
- Test connexion utilisateur
- Vérification pods Kubernetes
- Logs via `kubectl logs`
- Validation CI/CD via Jenkins

Tous les services sont fonctionnels :

`backend` `Running`

`frontend` `Running`

`postgres` `Running`

Conclusion

Ce projet a permis de :

- Mettre en œuvre une architecture microservices complète
- Automatiser entièrement un workflow DevOps
- Déployer une application distribuée sur Kubernetes
- Implémenter un pipeline CI/CD industriel

Les objectifs pédagogiques ont été atteints.

Le système est :

- Scalable
- Maintenable
- Automatisé
- Conforme aux bonnes pratiques DevOps

GitHub: <https://github.com/Adam01-i/SYSTEME-REPARTI>

DockerHub : <https://hub.docker.com/repositories/adam01tiret8>